UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE - UFRN
CENTRO DE ENSINO SUPERIOR DO SERIDÓ - CERES
DEPARTAMENTO DE COMPUTAÇÃO E TECNOLOGIA DCT

Henderson Emanuel Bernardo de Medeiros

Avaliação Comparativa de Modelos: Sistema Especialista, Árvore de Decisão e Ensemble Learning

Caicó - RN
2026

Henderson Emanuel Bernardo de Medeiros

Avaliação Comparativa de Modelos: Sistema Especialista, Árvore de Decisão e Ensemble Learning

Relatório de atividades apresentado como requisito para obtenção de nota na disciplina **Inteligência Artificial**, ofertada pela UFRN no período letivo 2026.2.

Professor: Prof. Dr. Thommas K. S. Flores

Universidade Federal do Rio Grande do Norte - UFRN
Centro de Ensino Superior do Seridó - CERES
Departamento de Computação e Tecnologia - DCT

Caicó - RN
2026

Índice

1 Introdução **4**

2 Protocolo Experimental **5**
2.1 Dispersão dos dados 5
2.2 Ambiente Computacional 5

3 Sistema Especialista **6**
3.1 Grafico de dispersão das variáves 6
3.2 Inferência do especialista e esforço humano 7

4 Árvore de Decisão **7**
4.1 Configuração e Hiperparâmetros 8
4.2 Desempenho da Árvore 8

5 Ensemble Learning **8**
5.1 Configurações e Hiperparâmetros do Ensemble 9
5.2 Desempenho do modelo 10
5.3 Importãncia das variáveis 10

6 Resultados **11**

7 Análise Comparativa e Discussão **11**

8 Considerações Finais **13**

9 Referência **14**

Lista de Figuras

1 Matriz de Correlação 4
2 Teste T 5
3 Dispersão 6
4 Regra 7
5 Overffiting da Árvore 7
6 Param. Árvore 8
7 Curva de erro 9
8 Param. Ensemble 9
9 Desemp. Ensemble 10
10 Importância das variáveis 10
11 Resultados 11
12 Generalização 12
13 Matriz de Confusão 12
14 Desemp. Computac 13

1 Introdução

Este trabalho tem o objetivo de fazer a comparação de modelos inteligência artificial, mais conhecidas como IA's, a partir de um conjunto de dados sobre desempenho acadêmico dos alunos. Esses dados serão divididos em treinamento, para a IA estudar e tentar criar uma generalização dos dados, e teste, onde a IA irá fazer uma prova para tentar aplicar essa generalização que foi feita na fase de treinamento.

Na estapa anterior do trabalho, foi feita uma analise e limpeza dos dados do nosso dataset, conseguido pelo site Kaggle, e a partir da análise e da natureza dos dados, foram escolhidas as melhores variáveis para achar o nosso target que seriam as notas dos alunos, ou seja, o problema se resume à: "Qual a nota daquele aluno de acordo com seus outros dados?". O objetivo das IA's é acertar com máximo desempenho as notas dos alunos visualizando apenas as variáveis que foram escolhidas na etapa da análise.

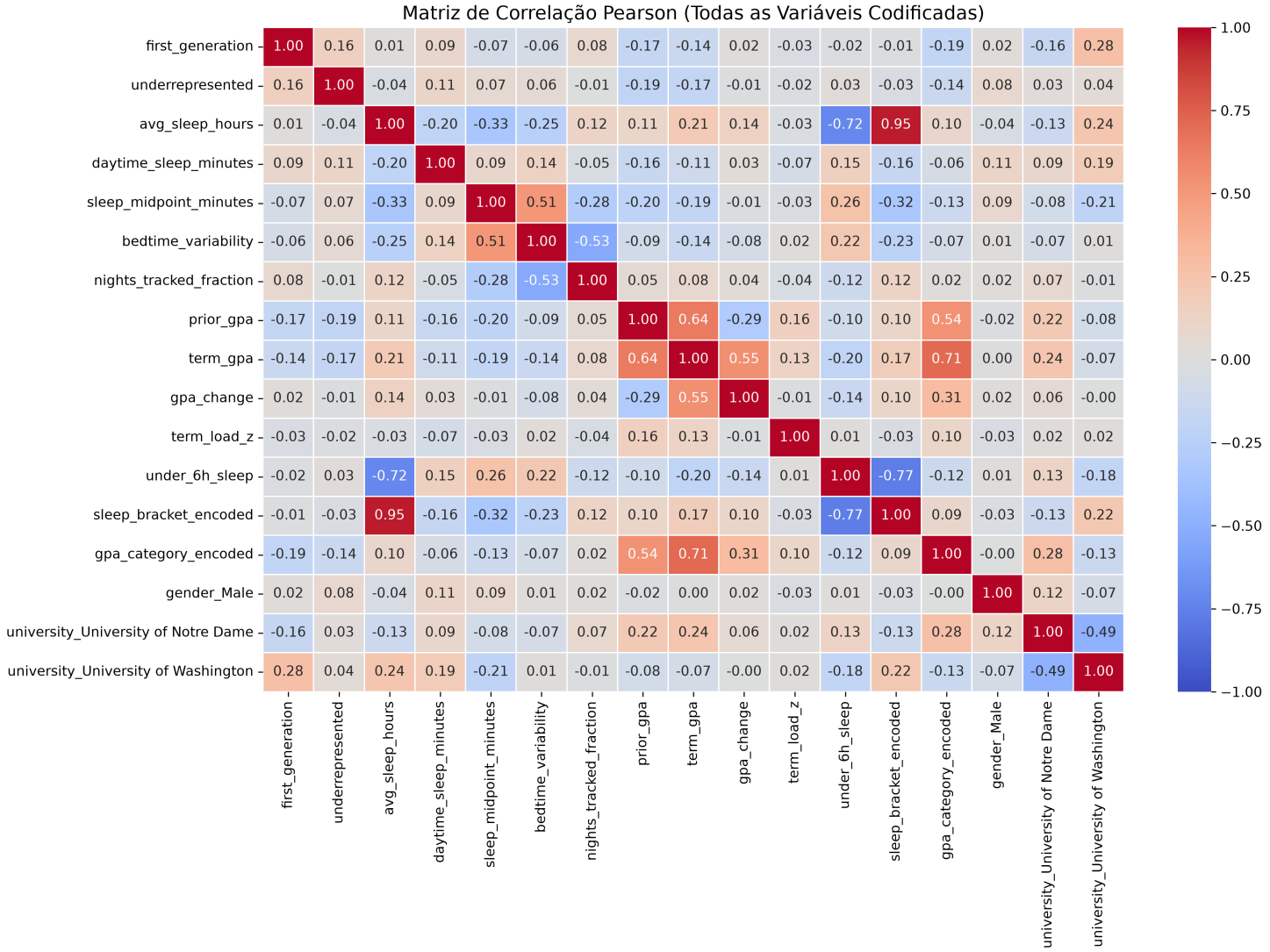


Figura 1: Matriz de Correlação

A partir de uma matriz de correleção(feito no trabalho anterior), achamos as 5 me-

lhores Variáveis, ou seja, aquelas variáveis, com maior valor significativo e importância para/que impactar na nota do aluno. Onde as variáveis foram: avg_sleep_hours(Tempo de sono em Horas),sleep_midpoint_minutes(Ponto médio de sono),prior_gpa(Nota do semestre anterior),underrepresented(Ser minoritário) e first_generation(Educação dos pais do aluno). foi a partir do Teste T, também do trabalho anterior, que confirmamos que as melhores variáveis da matriz realmente eram significativas.

```
=== TESTE T DE STUDENT (Alto vs Baixo Desempenho) ===
avg_sleep_hours           | p-valor: 0.0162 -> Significativo (p < 0.05)
sleep_midpoint_minutes    | p-valor: 0.0010 -> Significativo (p < 0.05)
prior_gpa                 | p-valor: 0.0000 -> Significativo (p < 0.05)
underrepresented          | p-valor: 0.0005 -> Significativo (p < 0.05)
first_generation          | p-valor: 0.0000 -> Significativo (p < 0.05)
```

Figura 2: Teste T

Além disso foi decidido com base na natureza dos dados, já que notas podem conter uma grande diversidade, que teriamos melhor desempenho trabalahndo com Classificação ao em vez de regressão. Assim os modelos teriam uma melhor performance nos testes e treinos que citaremos posteriormente.

2 Protocolo Experimental

Com as variáveis já selecionadas, como dito anteriomente, devemos dividirmos nosso dataset em treino, para o modelo aprender a generalizar esses dados, e testes, onde ele aplicará essa generalização para vermos como ele se saiu. Para isso devidimos todo o conjuto de dados em 80% para o treino e foco em apredizagem da generalização, e 20% para dos dados que serão aplicados na generalização aprendida anterior.

2.1 Dispersão dos dados

Utilizamos uma validação cruzada de 5, que siginifica que o dataset de treino era dividido sempre em 5, onde 4 deles era para o treino e 1 era para validação. repetindo o processo até que cada parte tenha sido usada para validação(isso apenas para o treino).

Também foi utilizado uma semente de valor 42, onde não diferencia em nada o modelo, mas os dados que seram sorteados para treino e teste seram afetados. Caso eu colocasse uma semente diferente de 42, teriamos um dataset com uma aleatoriedade diferente dos dados que teria no 42.

2.2 Ambiente Computacional

Além disso, para os teste foi usado um computador com processador Ryzen 5 da serie 5000, uma RAM de 16GB, no sistema operacional Ubuntu que é uma distro do linux.

Para o desenvolvimentos dos códigos foram utilizadas bibliotecas do python, como times, pandas seaborn, pandas, matplotlib, sklearn e tracemalloc.

3 Sistema Especialista

O modelo especialista, que é o modelo projetado pela propria visão do progamador sobre seus dados, é um modelo onde nós mesmos temos que visualizar a dispersão de cada variável e criar regras próprias a partir dessa visualização, onde essas regras serão jogadas para serem testadas no dataset e assim vermos se essas regras generalizam o nosso conjuto de dados.

3.1 Grafico de dispersão das variáves

As regras foram criadas partir de graficos de dispersão, onde comparamos todas as 5 variaveis uma com a outra e tentamos de alguma forma criar uma regra matemática que generalize aquela situação. Um exemplo é o gráfico abaixo que compara as horas de sono com a nota do aluno no semestre anterior, onde as bolinhas azuis são alunos com notas baixas e os verdes com notas altas.

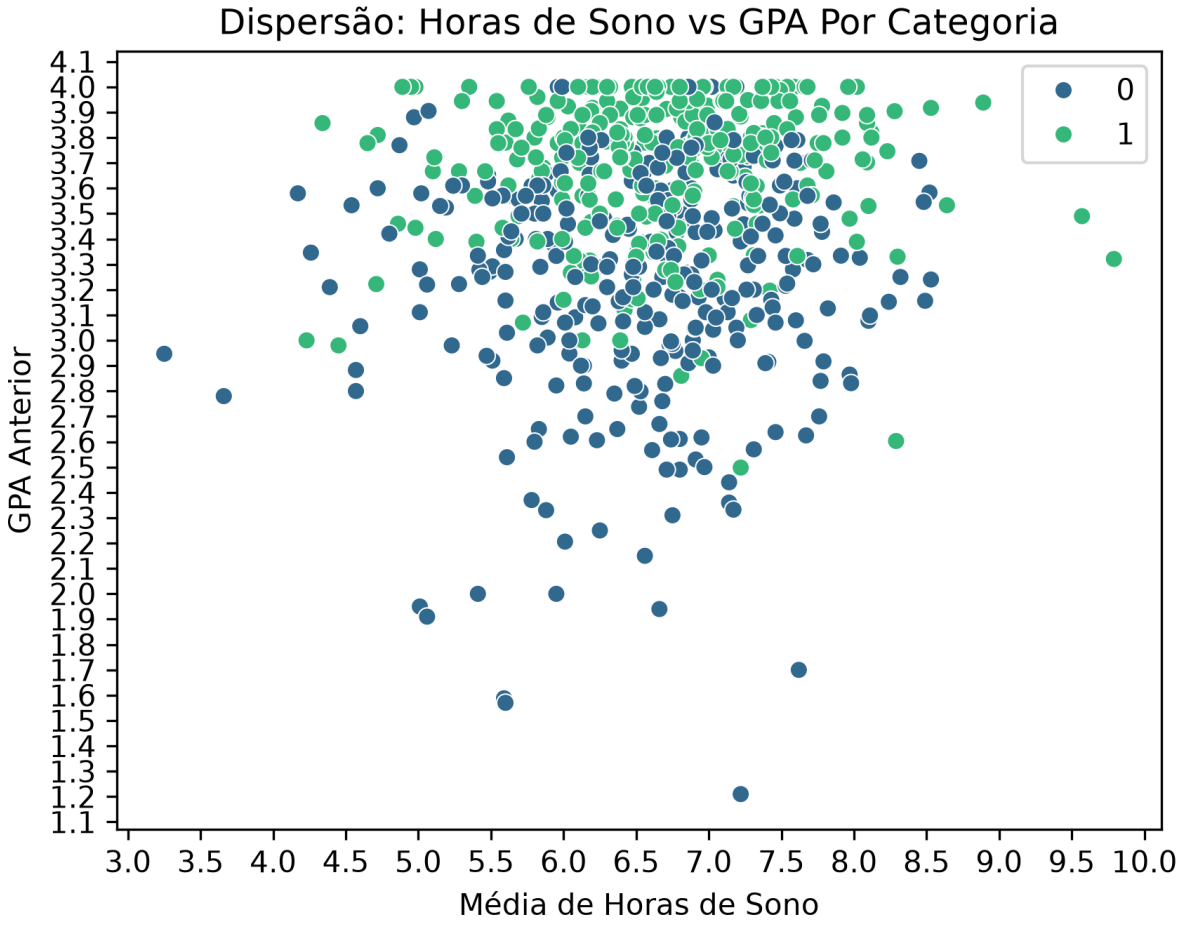


Figura 3: Dispersão

A partir desse gráfico podemos dizer que a gigantesca maioria dos alunos que tiveram uma nota anterior menor que 3, tiveram nota baixa duranrte o período de busca dos dados. Portante podemos criar a Regra da Figura abaixo:

```
expert_model.add_rule(Rule(
    rule_id="R1",
    condition_fn=lambda r: r['prior_gpa'] <= 3,
    target_class=0,
    rule_type="Relação",
    salience=1
))
```

Figura 4: Regra

Utilizamos esse procedimento para todas as 8 regras criadas.

3.2 Inferência do especialista e esforço humano

com base na biblioteca time, onde cronometramos todo o processo de aplicação das regras no dataset, tivemos um tempo total de 0.0033 segundos, um tempo medio de 0.0263 ms por cada amostra e uma Vazão(Throughput) de 37970.22 amostras/segundo.

Já para a criação das regras demorousse 2 horas ao todo, onde cada sessão, que foram 2, tiveram uma media entre 60 a 42 minutos, onde cada regra foi criada ali próximo a 15 minutos.

4 Árvore de Decisão

A árvore de decisão, que é basicamente um sistema especialista automatizado, onde o programador precisa apenas configurar seus hiperparâmetros que ela automaticamente fará as melhores regras possiveis para generalizar aqueles dados. é de suma importancia que configuremos elas com os melhores valores possiveis para que ela aprenda sem causar overfitting, como na figura abaixo, onde o modelo não aprendeu, mas decorou.

```
=== 1. Árvore Profunda (Demonstração de Overfitting) ===
Acurácia Treino: 1.0000 | Acurácia Teste: 0.6587
Gap de Generalização: 0.3413
```

Figura 5: Overffiting da Árvore

4.1 Configuração e Hiperparâmetros

Para achar a melhor árvore póssivel utilizamos de uma GidSearchCV, que é basicamnete um for que passa por cada valor e testa todas as possibilidades de configurações e hiperparâmetros e mostra a que mais se saiu bem. Utilizamos uma variedade de valores, podendo ser visualizados na Figura abaixo, para que fosse possível achar a real, melhor árvore.

```
parametros = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [2, 3, 4, 5, 6],
    'min_samples_split': [2, 4, 8, 16, 32],
    'min_samples_leaf': [1, 2, 4, 8, 16]
}
```

Figura 6: Param. Árvore

4.2 Desempenho da Árvore

Com as configurações já setados para que o GridSearch testasse todas as possibilidades, rodamos o treinamneto, e chegamos a uma Acurácia de treino de 77,40% e uma acurácia de teste de 75.40%, para essa configuração de entropia, max_depth = 2, min_samples_leaf = 1 e min_samples_split = 2.

Portanto com base nas acurácias, a acurácias conseguiu aprender bem a generalizar sem decorar, pois o desempenho antes e depois tiveram um gap de 2. um ótimo desempenho, conseguindo generalizar corretamente mais de 75% dos dados.

5 Ensemble Learning

No ensemble learn seguimos basicamente a mesma lógica que fizemos na ávore de decisão, mas utilizamos ramdom forests, que são varias árvores de decisão em conjunto achando uma resposta para o mesmo problema, e o gradient boosting, que são que são varias árvores, mas que um fica apenas com os erros da árvore passada tentando assim diminuir o erro.

É comum que quanto mais árvores trabalhando no problema, menor seja o erro, mas em certas ocasiões, se o modelo contiuar a adicionar mais e mais árvores, talvez, o erro suba, que é o que não queremos, e podemos ver isso na Figura abaixo, onde o erro no teste sobe um pouco depois de determinadp número de árvores

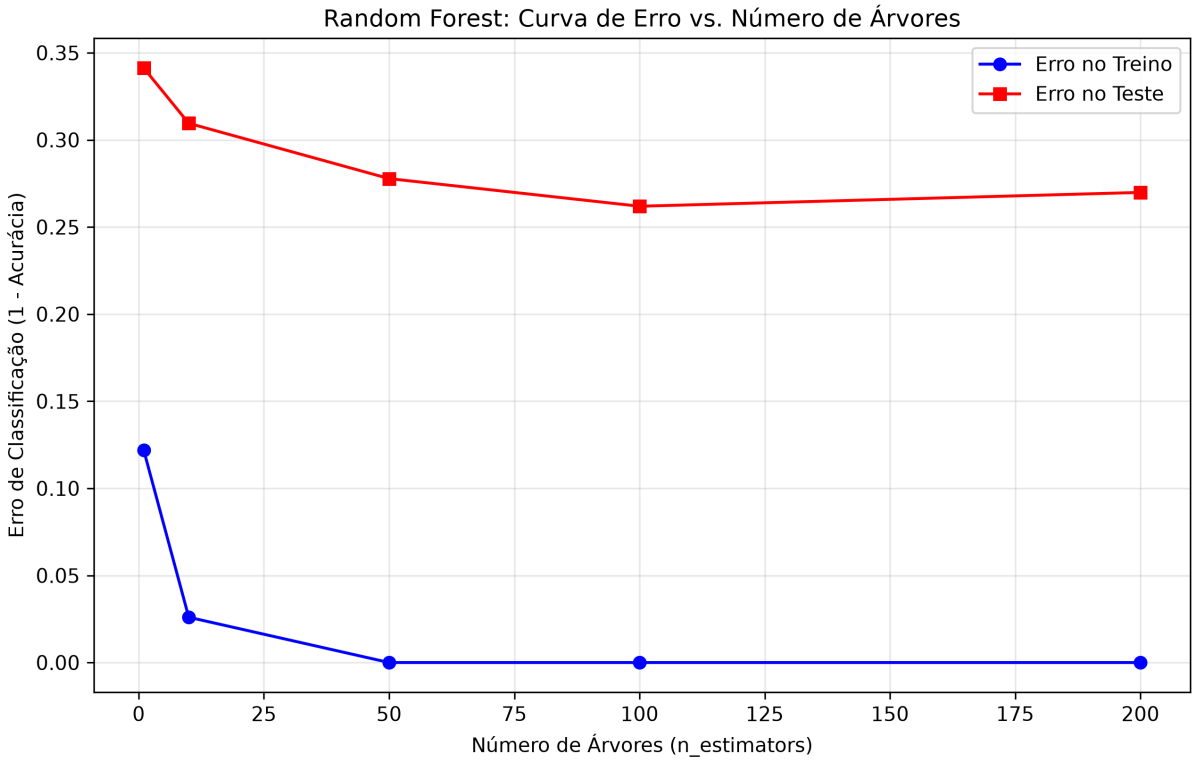


Figura 7: Curva de erro

5.1 Configurações e Hiperparâmetros do Ensemble

Para que não ocorra aumento de erro por parte de nosso modelo igual ocorreu no exemplo anterior. Fazemos do mesmo jeito da Árvore de decisão, utilizando o GridSearch, tanto no Random forest quanto no Gradient Booting. damos valores diversos para que o comando teste todas as possibilidades e nos dê a melhor.

```
param_grid_rf = {
    'n_estimators': [10, 25 , 50, 75, 100, 125, 150, 175, 200],
    'max_depth': [1, 2, 3, 5, 7, 11, 13],
    'min_samples_leaf': [1, 2, 3, 4]
}

# Grade para Gradient Boosting (Boosting)
param_grid_gb = {
    'n_estimators': [10, 25 , 50, 75, 100, 125, 150, 175, 200],
    'learning_rate': [0.001, 0.01, 0.1],
    'max_depth': [1, 2, 3, 5, 7, 11, 13]
}
```

Figura 8: Param. Ensemble

5.2 Desempenho do modelo

Como a árvore de decisão, os modelos de Ensemble ficaram entre 70% e 80%. O Random forest com configuração max_depth = 3, min_samples_leaf = 1 e n_estimators = 75; e o Gradient boosting com configuração learning_rate = 0.01, max_depth = 5 e n_estimators = 200; tiveram esse desempenho:

```
=== RESULTADOS DOS ENSEMBLES ===
Random Forest Melhor CV: 0.7740 | Acurácia Teste: 0.7143
Gradient Boosting Melhor CV: 0.7840 | Acurácia Teste: 0.7063
```

Figura 9: Desemp. Ensemble

podemos ver que tiveram um desempenho parecido, mas que o random forest conseguiu ganhar tendo um gap menor e uma accurácia final maior.

5.3 Importãncia das variáveis

Com base nos testes de cada modelo podemos também criar um gráfico que mostre a importânia de cada variável para acharmos a nota do aluno, ou seja, calssificar qual das 5 varáveis escolhidas tiveram mais peso na hora de descobrir se a nota era 0 ou 1, para os modelos, como random forest teve melhor desempenho foi feito um gráfico desse para ele:

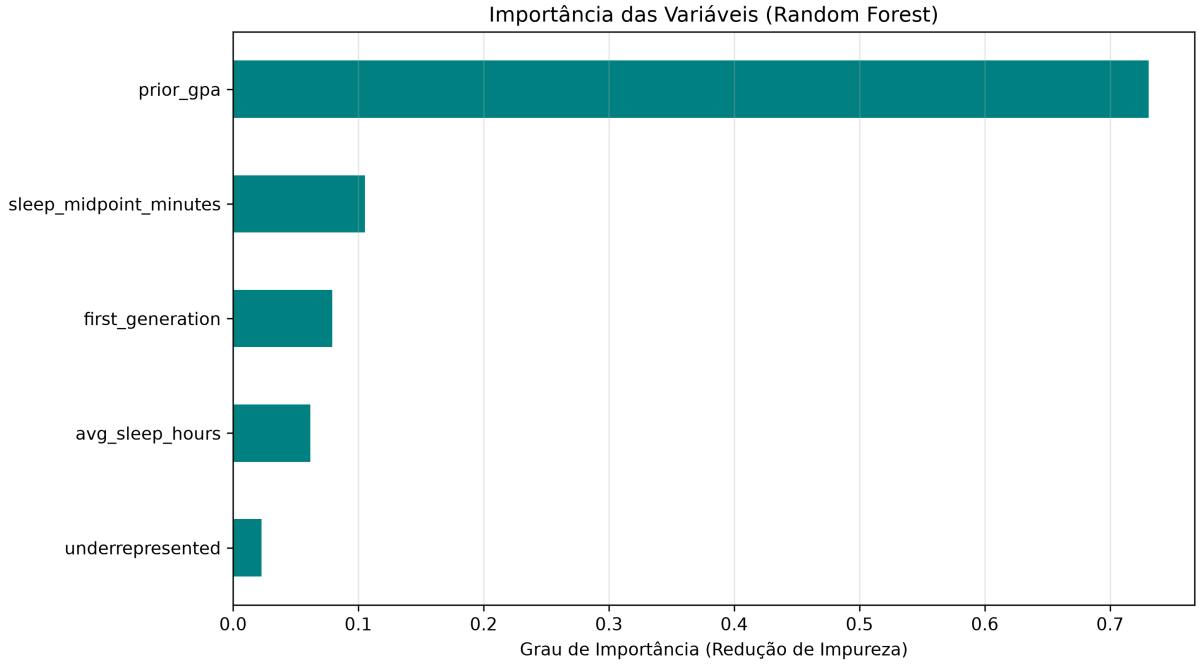


Figura 10: Importância das variáveis

Com base nos gráfico, podemos ver quer gigantesca parte dos dados podem ser achados apenas a partir do prio_gpa que é a nota do semestre anterior. ela tem um peso maior

que 0.7 enquanto todas as outras não passam nem de 0.11. isso quer dizer que em nosso dataset, podemos nos basear e confiar mais nessa variável.

6 Resultados

Trazendo todos esses 4 modelos ao qual trabalhamos, para uma comparação de métricas, temos esses resultados:

	Modelo	# Acurácia	# Precision	# Recall	# F1-Score	ROC-AUC	PR-AUC
	Missing: 0 (0%) Distinct: 4 (100%)	Missing: 0 (0%) Distinct: 4 (100%)	Missing: 0 (0%) Distinct: 4 (100%)	Missing: 0 (0%) Distinct: 3 (75%)	Missing: 0 (0%) Distinct: 4 (100%)	Missing: 0 (0%) Distinct: 4 (100%)	Missing: 0 (0%) Distinct: 4 (100%)
	Sistema Especialista 25% Árvore de Decisão 25% Random Forest 25% Other 25%	Min 0.65... Max 0.7...	Min 0.7288 Max 1.0	Min 0.31... Max 0.67...	Min 0.4762 Max 0.735	N/A 25% 0.8267 25% 0.8275 25% Other 25%	N/A 25% 0.8692 25% 0.8357 25% Other 25%
0	Sistema Especialista	0.6508	1.0	0.3125	0.4762	N/A	N/A
1	Árvore de Decisão	0.754	0.8113	0.6719	0.735	0.8267	0.8692
2	Random Forest	0.7143	0.75	0.6562	0.7	0.8275	0.8357
3	Gradient Boosting	0.7063	0.7288	0.6719	0.6992	0.7826	0.8158

Figura 11: Resultados

Com base nesses resultados temos que a única árvore de decisão teve o maior desempenho em todas as métricas, comparando com os 2 modelos de ensemble learning e o modelo especialista. Obteve a melhor Acurácia global (75,4%); a melhor Precisão (81,13%), que são basicamente os verdadeiros positivos e falsos que ele acertou; o melhor F1-Score (73,5%), que é a média entre a Precisão e o Recall; e curvas ROC e PR ambos próximos de 0.87.

O Sistema Especialista apresentou as piores métricas, sinalizando um severo underfitting. A sua Acurácia foi a mais baixa (65,08%), um Recall incrivelmente fraco (31,25%). E Apesar de a sua Precisão ser perfeita (100%), isto é um efeito métricas estatísticas: significa que as poucas vezes em que o sistema arriscou classificar um estudante na Classe Positiva (Alto Desempenho), ele acertou, mas na esmagadora maioria dos casos, o sistema foi incapaz de identificar os alunos de alto desempenho (daí o recall baixo e o pior F1-Score do teste: 0.4762). Além disso, ROC-AUC e PR-AUC não puderam ser calculadas para este modelo porque os sistemas especialistas tradicionais geram saídas 0 ou 1, e não pontuações de probabilidade contínuas.

7 Análise Comparativa e Discussão

Com base nessas generalização de cada modelo:

Modelo	# Acc Treino	# Acc Teste	# Gap (...	CV Média (5-Fold)	CV Desvio Padrão
Missing: 0 (0%)	Missing: 0 (0%)	Missing: 0%	Missing: 0%	Missing: 0 (0%)	Missing: 0 (0%)
Distinct: 4 (100%)	Disti... 4 (100%)	Distinct: 100%	Distin...100%	Distinct: 3 (75%)	Distinct: 4 (100%)
Sistema Especialista 25%	Min 0.... Max 0....	0.6508 0.754	0.02 0.1817	0.774 50%	N/A 25%
Árvore de Decisão 25%				N/A 25%	0.0372 25%
Random Forest 25%				0.784 25%	0.0314 25%
Other 25%					Other 25%
Sistema Especialista	0.682	0.6508	0.0312	N/A	N/A
Árvore de Decisão	0.774	0.754	0.02	0.774	0.0372
Random Forest	0.788	0.7143	0.0737	0.774	0.0314
Gradient Boosting	0.888	0.7063	0.1817	0.784	0.008

Figura 12: Generalização

Portanto, a árvore deve maior desempenho e que melhor generalizou os dados. E em comparação com os ensemble, temos pouca variância, mas ainda assim foi a que teve melhor desempenho. Podemos ver isso também pelo gráfico de confusão, onde a única árvore foi a que mais acertou.

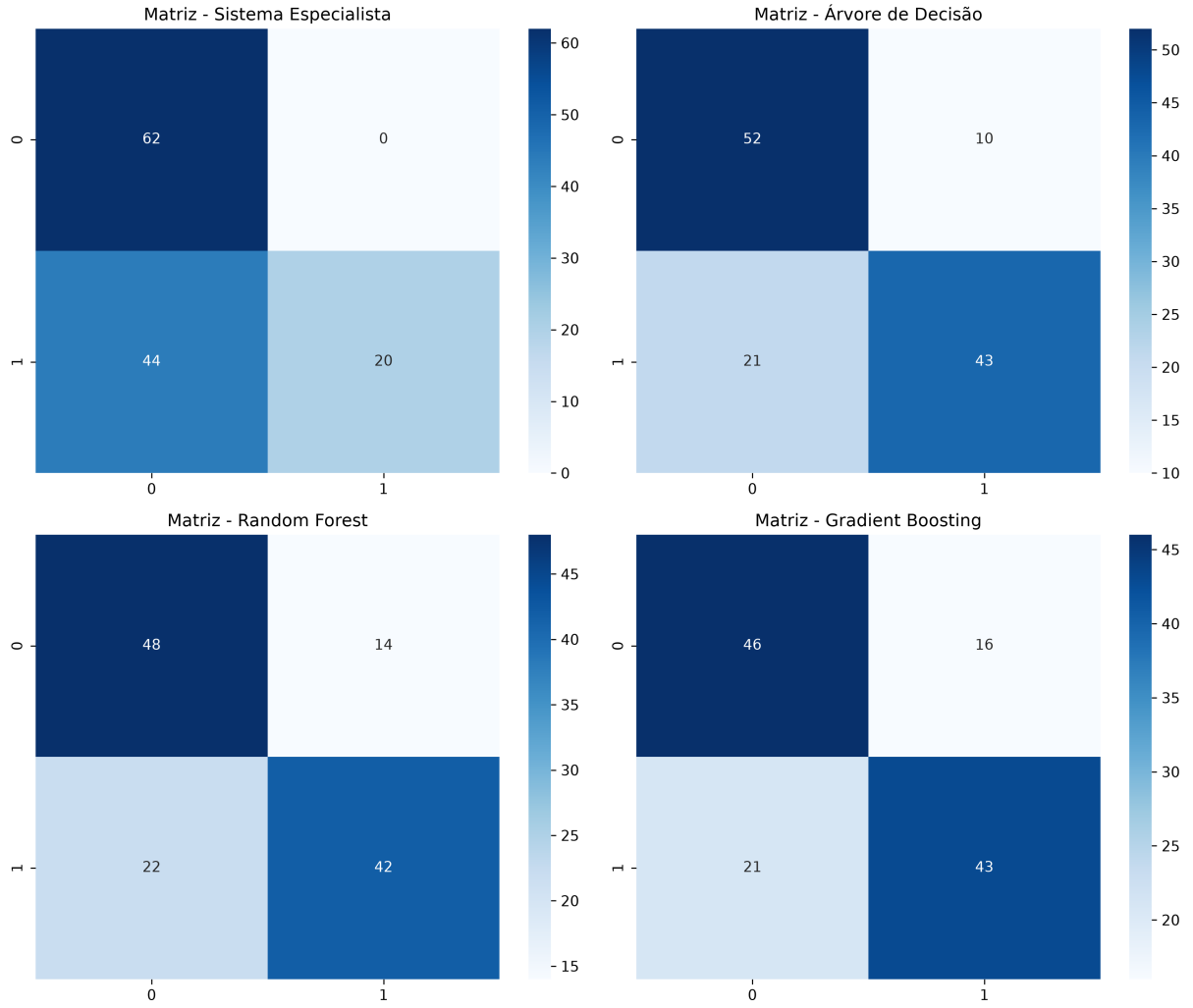
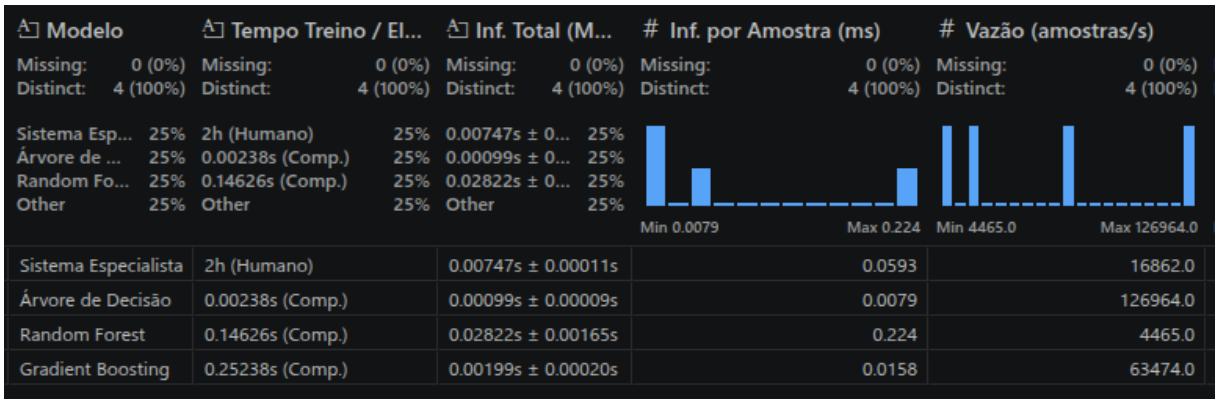


Figura 13: Matriz de Confusão

A ávore tambem é que dtém melhor desempnho computacional, pois é que teve mais acertos e não precisou ficar criando várias árvores, pois quanto mais árvores, menor o desempenho computacional, qué o que acontece nos ensemble, então mesmo que criassemos centenas de árvores apenas para aumentar a acurácia em 1% por exemplo, seria um esforço inútil.

O modelo especialista foi o pior, além de ter sido oq ue mais demora a ser feito, pois ele é o único que não é feito automáticamente, mas sim pelo próprio programador. na figura baixo podemos ver a baixa eficiência dele no nosso experimento:



Sistema Especialista	2h (Humano)	0.00747s ± 0.00011s	0.0593	16862.0
Árvore de Decisão	0.00238s (Comp.)	0.00099s ± 0.00009s	0.0079	126964.0
Random Forest	0.14626s (Comp.)	0.02822s ± 0.00165s	0.224	4465.0
Gradient Boosting	0.25238s (Comp.)	0.00199s ± 0.00020s	0.0158	63474.0

Figura 14: Desemp. Computac

8 Considerações Finais

Com esse trabalho apredemos que nem sempre o maior e mais robusto é melhor. tudo depende e vai de caso para caso. no nosso exemplo por exemplo, uma única árvore foi melhor que centenas. Além disso, um dataset bem limpo, não quer dizer que seu modelo vá conseguir aprender tudo ou generalizar tudo, pois como vimos, escolhemos 4 variáveis, que lá na análise davam parecidas com a prio_gpa, mas quando colocamos para os modelos aprenderem, eles em sua gigantesca maioria, apenas dividiram genelarizaram o dataset todo apenas com base na prior_gpa, que foi o que vimos no gráfico de importâmcia.

Como nota não depende apenas de sono, que era o que era pra ser o objetivo do dataset, mas sim de diversas outras variáveis, estourando o escopo para variáveis que não tinham no dataset e que não utilizamos por esse motivo, então generalizar algo tão amplo, que é o "o que faz o aluno tirar notas boas?"com um dataset tão pequeno e com tão poucas diversidade de colunas, é algo muito complicado, principalmente para o modelo especialista. Portanto, caso o dataset fosse mais amplo e maior, provavelmente deixaríamos de ficar presos entre os 70% e 80% de acurácia, coisa que vimos na ávore e nos modelos ensemble, e talvez saltariamospara os 90+.

Portanto, também aprendemos que os modelos de Árvore e florestas conseguem genrealizar melhor algo amplo que tenham uma dispersão muito grande, e que não tenham divisão clara aos olhos, para generalizarmos apenas com os olhos igual fazemos com o especialista.

9 Referência

https://github.com/HendsMede1r0s/AnaliseDeDados